2.4.4 Binary Indexed (Fenwick) Tree

Fenwick Tree—also known as **Binary Indexed Tree** (BIT)—were invented by *Peter M. Fenwick* in 1994 [18]. In this book, we will use the term Fenwick Tree as opposed to BIT in order to differentiate with the standard *bit manipulations*. The Fenwick Tree is a useful data structure for implementing *dynamic cumulative frequency tables*. Suppose we have[29] test scores of `m = 11` students `f = {2,4,5,5,6,6,6,7,7,8,9}` where the test scores are *integer values* ranging from `[1..10]`. Table 2.1 shows the frequency of each individual test score ∈ `[1..10]` and the cumulative frequency of test scores ranging from `[1..i]` denoted by `cf[i]`—that is, the sum of the frequencies of test scores `1, 2, ..., i`.

Index/ Score	Frequency `f`	Cumulative Frequency `cf`	Short Comment
0	-	-	Index 0 is ignored (as the sentinel value).
1	0	0	`cf[1] = f[1] = 0.`
2	1	1	`cf[2] = f[1] + f[2] = 0 + 1 = 1.`
3	0	1	`cf[3] = f[1] + f[2] + f[3] = 0 + 1 + 0 = 1.`
4	1	2	`cf[4] = cf[3] + f[4] = 1 + 1 = 2.`
5	2	4	`cf[5] = cf[4] + f[5] = 2 + 2 = 4.`
6	3	7	`cf[6] = cf[5] + f[6] = 4 + 3 = 7.`
7	2	9	`cf[7] = cf[6] + f[7] = 7 + 2 = 9.`
8	1	10	`cf[8] = cf[7] + f[8] = 9 + 1 = 10.`
9	1	11	`cf[9] = cf[8] + f[9] = 10 + 1 = 11.`
10	0	11	`cf[10] = cf[9] + f[10] = 11 + 0 = 11.`

Table 2.1: Example of a Cumulative Frequency Table

The cumulative frequency table can also be used as a solution to the Range Sum Query (RSQ) problem mentioned in **Exercise 2.4.3.2***. It stores `RSQ(1, i)` $\forall i \in$ `[1..n]` where `n` is the largest integer index/score[30]. In the example above, we have `n = 10`, `RSQ(1, 1) = 0`, `RSQ(1, 2) = 1`, ..., `RSQ(1, 6) = 7`, ..., `RSQ(1, 8) = 10`, ..., and `RSQ(1, 10) = 11`. We can then obtain the answer to the RSQ for an arbitrary range `RSQ(i, j)` when $i \neq 1$ by subtracting `RSQ(1, j) - RSQ(1, i - 1)`. For example, `RSQ(4, 6) = RSQ(1, 6) - RSQ(1, 3) = 7 - 1 = 6`.

If the frequencies are *static*, then the cumulative frequency table as in Table 2.1 can be computed efficiently with a simple $O(n)$ loop. First, set `cf[1] = f[1]`. Then, for `i` ∈ `[2..n]`, compute `cf[i] = cf[i - 1] + f[i]`. This will be discussed further in Section 3.5.2. However, when the frequencies are frequently updated (increased or decreased) and the RSQs are frequently asked afterwards, it is better to use a *dynamic* data structure.

Instead of using a Segment Tree to implement a *dynamic* cumulative frequency table, we can implement the *far simpler* Fenwick Tree instead (compare the source code for both implementations, provided in this section and in the previous Section 2.4.3). This is perhaps one of the reasons why the Fenwick Tree is currently included in the IOI syllabus [20]. Fenwick Tree operations are also extremely efficient as they use fast bit manipulation techniques (see Section 2.2).

In this section, we will use the function `LSOne(i)` (which is actually `(i & (-i))`) extensively, naming it to match its usage in the original paper [18]. In Section 2.2, we have seen that the operation `(i & (-i))` produces the first Least Significant One-bit in `i`.

[29] The test scores are shown in sorted order for simplicity, they do not have to be sorted.

[30] Please differentiate `m` = the number of data points and `n` = the largest integer value among the `m` data points. The meaning of `n` in Fenwick Tree is a bit different compared to other data structures in this book.

The Fenwick Tree is typically implemented as an array (we use a `vector` for size flexibility). The Fenwick Tree is a tree that is indexed by the *bits* of its *integer* keys. These integer keys fall within the fixed range `[1..n]`—skipping[31] index 0. In a programming contest environment, `n` can approach $\approx 1M$ so that the Fenwick Tree covers the range `[1..`$1M$`]`—large enough for many practical (contest) problems. In Table 2.1 above, the scores `[1..10]` are the integer keys in the corresponding array with size `n = 10` and `m = 11` data points.

Let the name of the Fenwick Tree array be `ft`. Then, the element at index `i` is responsible for elements in the range `[i-LSOne(i)+1..i]` and `ft[i]` stores the cumulative frequency of elements {`i-LSOne(i)+1, i-LSOne(i)+2, i-LSOne(i)+3, .., i`}. In Figure 2.11, the value of `ft[i]` is shown in the circle above index `i` and the range `[i-LSOne(i)+1..i]` is shown as a circle and a bar (if the range spans more than one index) above index `i`. We can see that `ft[4] = 2` is responsible for range `[4-4+1..4] = [1..4]`, `ft[6] = 5` is responsible for range `[6-2+1..6] = [5..6]`, `ft[7] = 2` is responsible for range `[7-1+1..7] = [7..7]`, `ft[8] = 10` is responsible for range `[8-8+1..8] = [1..8]` etc[32].

With such an arrangement, if we want to obtain the cumulative frequency between `[1..b]`, i.e. `rsq(b)`, we simply add `ft[b]`, `ft[b']`, `ft[b'']`, ... until index `b`^{i} is 0. This sequence of indices is obtained via subtracting the Least Significant One-bit via the bit manipulation expression: `b' = b - LSOne(b)`. Iteration of this bit manipulation effectively *strips off* the least significant one-bit of `b` at each step. As an integer `b` only has $O(\log b)$ bits, `rsq(b)` runs in $O(\log n)$ time when `b = n`. In Figure 2.11, `rsq(6) = ft[6] + ft[4] = 5 + 2 = 7`. Notice that indices 4 and 6 are responsible for range `[1..4]` and `[5..6]`, respectively. By combining them, we account for the entire range of `[1..6]`. The indices 6, 4, and 0 are related in their binary form: `b =` 6_{10} `=` $(1\underline{1}0)_2$ can be transformed to `b' =` 4_{10} `=` $(\underline{1}00)_2$ and subsequently to `b'' =` 0_{10} `=` $(000)_2$.

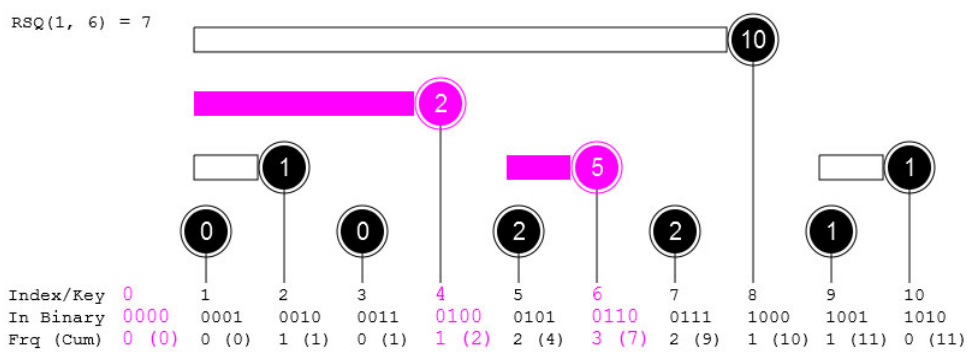

Figure 2.11: Example of `rsq(6)`

With `rsq(b)` available, obtaining the cumulative frequency between two indices `[a..b]` where `a != 1` is simple, just evaluate `rsq(a, b) = rsq(b) - rsq(a - 1)`. For example, if we want to compute `rsq(4, 6)`, we can simply return `rsq(6) - rsq(3) = (5+2) - (0+1) = 7 - 1 = 6`. Again, this operation runs in $O(2 \times \log b) \approx O(\log n)$ time when `b = n`. Figure 2.12 displays the value of `rsq(3)`.

When updating the value of the element at index `k` by adjusting its value by `v` (note that `v` can be either positive or negative), i.e. calling `adjust(k, v)`, we have to update `ft[k]`, `ft[k']`, `ft[k'']`, ... until index `k`^{i} exceeds n. This sequence of indices are obtained

[31] We have chosen to follow the original implementation by [18] that ignores index 0 to facilitate an easier understanding of the bit manipulation operations of Fenwick Tree. Note that index 0 has no bit turned on. Thus, the operation `i +/- LSOne(i)` simply returns `i` when `i = 0`. Index 0 is also used as the terminating condition in the `rsq` function.

[32] In this book, we will not detail why this arrangement works and will instead show that it allows for efficient $O(\log n)$ update and RSQ operations. Interested readers are advised to read [18].

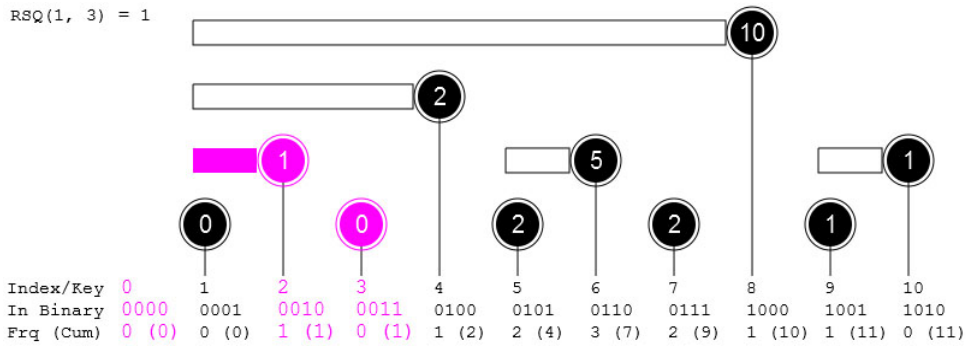

Figure 2.12: Example of `rsq(3)`

via this similar iterative bit manipulation expression: `k' = k + LSOne(k)`. Starting from any integer `k`, the operation `adjust(k, v)` will take at most $O(\log n)$ steps until `k` $>$ `n`. In Figure 2.13, `adjust(5, 1)` will affect (add +1 to) `ft[k]` at indices `k` $= 5_{10} = (101)_2$, `k'` $= (101)_2$ `+` $(00\underline{1})_2 = (110)_2 = 6_{10}$, and `k''` $= (110)_2$ `+` $(0\underline{1}0)_2 = (1000)_2 = 8_{10}$ via the expression given above. Notice that if you project a line upwards from index 5 in Figure 2.13, you will see that the line indeed *intersects* the ranges under the responsibility of index 5, index 6, and index 8.

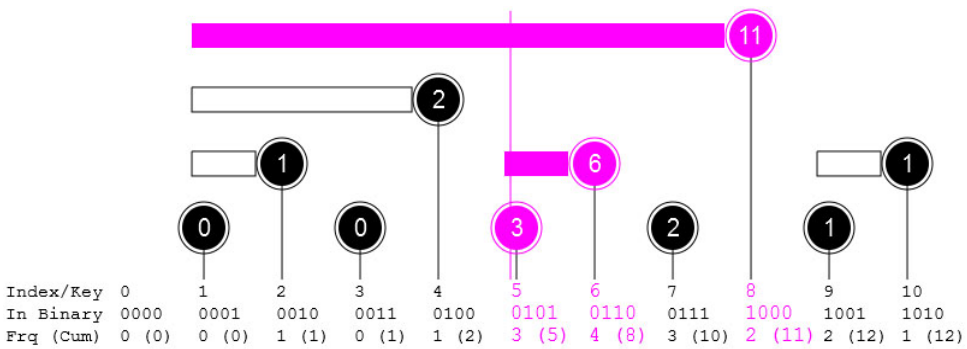

Figure 2.13: Example of `adjust(5, 1)`

In summary, Fenwick Tree supports both RSQ and update operations in just $O(n)$ space and $O(\log n)$ time given a set of m integer keys that ranges from `[1..`n`]`. This makes Fenwick Tree an ideal data structure for solving *dynamic* RSQ problems on with discrete arrays (the *static* RSQ problem can be solved with simple $O(n)$ pre-processing and $O(1)$ per query as shown earlier). Our *short* C++ implementation of a basic Fenwick Tree is shown below.

```
class FenwickTree {
private: vi ft;                // recall that vi is: typedef vector<int> vi;
public: FenwickTree(int n) { ft.assign(n + 1, 0); }    // init n + 1 zeroes
  int rsq(int b) {                                     // returns RSQ(1, b)
    int sum = 0; for (; b; b -= LSOne(b)) sum += ft[b];
    return sum; }                                // note: LSOne(S) (S & (-S))
  int rsq(int a, int b) {                              // returns RSQ(a, b)
    return rsq(b) - (a == 1 ? 0 : rsq(a - 1)); }
  // adjusts value of the k-th element by v (v can be +ve/inc or -ve/dec)
  void adjust(int k, int v) {                      // note: n = ft.size() - 1
    for (; k < (int)ft.size(); k += LSOne(k)) ft[k] += v; }
};
```

```
int main() {
  int f[] = { 2,4,5,5,6,6,6,7,7,8,9 };                    // m = 11 scores
  FenwickTree ft(10);           // declare a Fenwick Tree for range [1..10]
      // insert these scores manually one by one into an empty Fenwick Tree
  for (int i = 0; i < 11; i++) ft.adjust(f[i], 1);    // this is O(k log n)
  printf("%d\n", ft.rsq(1, 1));  // 0 => ft[1] = 0
  printf("%d\n", ft.rsq(1, 2));  // 1 => ft[2] = 1
  printf("%d\n", ft.rsq(1, 6));  // 7 => ft[6] + ft[4] = 5 + 2 = 7
  printf("%d\n", ft.rsq(1, 10)); // 11 => ft[10] + ft[8] = 1 + 10 = 11
  printf("%d\n", ft.rsq(3, 6));  // 6 => rsq(1, 6) - rsq(1, 2) = 7 - 1
  ft.adjust(5, 2);               // update demo
  printf("%d\n", ft.rsq(1, 10)); // now 13
} // return 0;
```

Visualization: `www.comp.nus.edu.sg/~stevenha/visualization/bit.html`

Source code: `ch2_10_fenwicktree_ds.cpp/java`

Exercise 2.4.4.1: Just a simple exercise of the two basic bit-manipulation operations used in the Fenwick Tree: What are the values of `90 - LSOne(90)` and `90 + LSOne(90)`?

Exercise 2.4.4.2: What if the problem that you want to solve includes an element at integer key 0? Recall that the standard integer key range in our library code is is `[1..n]` and that this implementation cannot use index 0 since it is used as the terminating condition of `rsq`.

Exercise 2.4.4.3: What if the problem that you want to solve uses non-integer keys? For example, what if the test scores shown in Table 2.1 above are `f = {5.5, 7.5, 8.0, 10.0}` (i.e. allowing either a 0 or a 5 after the decimal place)? What if the test scores are `f = {5.53, 7.57, 8.10, 9.91}` (i.e. allowing for two digits after the decimal point)?

Exercise 2.4.4.4: The Fenwick Tree supports an additional operation that we have decided to leave as an exercise to the reader: Find the smallest index with a given cumulative frequency. For example, we may need to determine the minimum index/score `i` in Table 2.1 such that there are at least 7 students covered in the range `[1..i]` (index/score 6 in this case). Implement this feature.

Exercise 2.4.4.5*: Solve this dynamic RSQ problem: UVa 12086 - Potentiometers using *both* a Segment Tree and Fenwick Tree. Which solution is easier to produce in this case? Also see Table 2.2 for a comparison between these two data structures.

Exercise 2.4.4.6*: Extend the 1D Fenwick Tree to 2D!

Exercise 2.4.4.7*: Fenwick Trees are normally used for point update and range (sum) query. Show how to use a Fenwick Tree for *range update* and point queries. For example, given lots of intervals with small ranges (from 1 to at most 1 million) determine the number of intervals encompassing index i.

Profile of Data Structure Inventors

Peter M. Fenwick is a Honorary Associate Professor in the University of Auckland. He invented the Binary Indexed Tree in 1994 [18] as "cumulative frequency tables of arithmetic compression". The BIT has since been included in the IOI syllabus [20] and used in many contest problems for its efficient yet easy to implement data structure.

Feature	Segment Tree	Fenwick Tree
Build Tree from Array	$O(n)$	$O(m \log n)$
Dynamic RMin/MaxQ	OK	Very limited
Dynamic RSQ	OK	OK
Query Complexity	$O(\log n)$	$O(\log n)$
Point Update Complexity	$O(\log n)$	$O(\log n)$
Length of Code	Longer	Shorter

Table 2.2: Comparison Between Segment Tree and Fenwick Tree

Programming exercises that use the data structures discussed and implemented:

- Graph Data Structures Problems
  1. **UVa 00599 - The Forrest for the Trees *** ($v-e$ = number of connected components, keep a `bitset` of size 26 to count the number of vertices that have some edge. Note: Also solvable with Union-Find)
  2. **UVa 10895 - Matrix Transpose *** (transpose adjacency list)
  3. UVa 10928 - My Dear Neighbours (counting out degrees)
  4. UVa 11550 - Demanding Dilemma (graph representation, incidence matrix)
  5. **UVa 11991 - Easy Problem from ... *** (use the idea of an Adj List)

  Also see: More graph problems in Chapter 4
- Union-Find Disjoint Sets
  1. **UVa 00793 - Network Connections *** (trivial; application of disjoint sets)
  2. UVa 01197 - The Suspects (LA 2817, Kaohsiung03, Connected Components)
  3. UVa 10158 - War (advanced usage of disjoint sets with a nice twist; memorize list of enemies)
  4. UVa 10227 - Forests (merge two disjoint sets if they are consistent)
  5. **UVa 10507 - Waking up brain *** (disjoint sets simplifies this problem)
  6. UVa 10583 - Ubiquitous Religions (count disjoint sets after all unions)
  7. UVa 10608 - Friends (find the set with the largest element)
  8. UVa 10685 - Nature (find the set with the largest element)
  9. **UVa 11503 - Virtual Friends *** (maintain set attribute (size) in rep item)
  10. UVa 11690 - Money Matters (check if total money from each member is 0)
- Tree-related Data Structures
  1. UVa 00297 - Quadtrees (simple quadtree problem)
  2. UVa 01232 - SKYLINE (LA 4108, Singapore07, a simple problem if input size is small; but since $n \leq 100000$, we have to use a Segment Tree; note that this problem is not about RSQ/RMQ)
  3. **UVa 11235 - Frequent Values *** (range maximum query)
  4. UVa 11297 - Census (Quad Tree with updates or use 2D segment tree)
  5. UVa 11350 - Stern-Brocot Tree (simple tree data structure question)
  6. **UVa 11402 - Ahoy, Pirates *** (segment tree with *lazy* updates)
  7. UVa 12086 - Potentiometers (LA 2191, Dhaka06; pure dynamic range sum query problem; solvable with Fenwick Tree or Segment Tree)
  8. ***UVa 12532 - Interval Product **** (clever usage of Fenwick/Segment Tree)

  Also see: DS as part of the solution of harder problems in Chapter 8